Security Optimization Assessment — Defender for Identity

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Security Optimization Assessment: Defender for Identity 5
- What you get . . . 5
- What this delivers . . . 5
- The standard deployment model . . . 6
- Outcome . . . 6
- Scope boundary . . . 6

1. Solution Overview — Security Optimization Assessment: Defender for Identity 6
- What this assessment is . . . 6
- How Defender for Identity fits together (what we score) . . . 6
- How the score works . . . 7
- What's in scope . . . 7
- What's out of scope (be honest with buyers) . . . 7
- Authentication and connections used . . . 8
- The standard deployment model . . . 8

2. Requirements — Security Optimization Assessment: Defender for Identity 8
- R1. Defender for Identity is provisioned (even partially) . . . 8
- R2. Read-only Active Directory access to enumerate the identity inventory . . . 9
- R3. Graph read access for identity Secure Score and advanced hunting . . . 9
- R4. Portal read access to MDI sensor health, DSA, and honeytoken configuration . . . 9
- R5. The expected server inventory is defined . . . 9
- R6. A named best-practice baseline for scoring . . . 10
- R7. Secure destination for the findings report . . . 10

Global Requirements — Assessments (category-level) 10
- G1. The workload/tenant being assessed . . . 10
- G2. Read-only assessor access (least privilege) . . . 10
- G3. Microsoft Graph / Az / management API access . . . 10
- G4. Licensing for the products in scope . . . 11
- G5. Scope definition and a point-in-time baseline . . . 11
- G6. Best-practice reference baseline . . . 11
- G7. Output handling and confidentiality . . . 11
- G8. Remediation ownership (for the paired remediation) . . . 11

3. Implementation — Security Optimization Assessment: Defender for Identity 12
- Stage 0 — Confirm scope and baseline (Assess) . . . 12
- Stage 1 — Establish read-only access (Assess) . . . 12
- Stage 2 — Validate the read (Pilot the read) . . . 12

Stage 3 — Discover the configuration (Assess) . . . 13
Stage 4 — Reconcile sensor coverage (Assess) . . . 13
Stage 5 — Score against best practice (Assess) . . . 13
Stage 6 — Generate the findings report (Validate → deliver) . . . 13
Stage 7 — Review and hand off (Expand) . . . 14
5. Testing — Security Optimization Assessment: Defender for Identity 14
Gate 1 — Read-only access proven (run `SOAMDI-Validate.ps1`) . . . 14
Gate 2 — The DC inventory is complete (the denominator) . . . 14
Gate 3 — Discovery completeness (run `SOAMDI-Discovery.ps1`) . . . 14
Gate 4 — Coverage names the blind spots (run `SOAMDI-Coverage.ps1`) . . . 15
Gate 5 — Scoring is defensible (run `SOAMDI-Assess.ps1`) . . . 15
Gate 6 — Report reconciles with the portal (run `SOAMDI-Report.ps1`) . . . 15
Gate 7 — Repeatability (trend) . . . 15
6. Troubleshooting — Security Optimization Assessment: Defender for Identity 15
`Get-ADDomainController` returns fewer DCs than expected . . . 16
AD module not available . . . 16
Sensor list / DSA / honeytoken data comes back empty from the script . . . 16
`Connect-MgGraph` or advanced hunting fails . . . 16
Coverage shows 100% but the owner suspects blind DCs . . . 16
An un-sensored DC in the report is actually being decommissioned . . . 16
Validate flags a write scope present . . . 17
Report `verify` items not confirmed before delivery (Gate 6 risk) . . . 17
7. Best Practices — Security Optimization Assessment: Defender for Identity 17
Keep it strictly read-only . . . 17
The denominator is everything . . . 17
Confirm what has no API — don't drop it . . . 18
Interpret each area honestly . . . 18
Make the numbers defensible . . . 18
Hand off well . . . 18
Protect the output . . . 18
Where to go next (adjacent catalog areas) . . . 18
8. Deliverables Summary — Security Optimization Assessment: Defender for Identity 19
Documentation . . . 19
Automation artifacts (all read-only) . . . 19
What the buyer receives . . . 20
Outcome . . . 20

Appendix A — PowerShell Scripts 21

Executive Summary — Security Optimization Assessment: Defender for Identity

White-label · Assessments

Microsoft Defender for Identity (MDI) protects your on-premises identity fabric — but only where a **sensor** is actually installed and healthy. Teams deploy MDI to "most" domain controllers, wire up a directory-service account, and move on — leaving the questions that decide whether it works unanswered: is a sensor on **every** domain controller (and on **AD FS**, **AD CS**, and **Entra Connect** where present), or are there blind DCs an attacker can pivot through; is the **directory-service account (DSA)** configured (ideally as a **gMSA**) so MDI can enrich identities; are **honeytoken** accounts planted and tagged to trip intruders; is your **Identity Security Posture Management (ISPM)** backlog being worked; and are detections **tuned** (exclusions sane, action accounts scoped) rather than noisy or blind. A deployed sensor on some DCs is not identity coverage — and identity is the terrain every serious intrusion crosses.

This solution is a **read-only Security Optimization Assessment**. It discovers your MDI configuration, **scores it against Microsoft best practice**, and produces a **prioritized, risk-ranked findings report** — sensor coverage on DCs/AD FS/AD CS, directory-service account posture, honeytoken & ISPM posture, and detection tuning — so you know, with evidence, exactly what to fix and in what order. It **changes nothing**: reads come from read-only Graph/advanced-hunting scopes and read-only Active Directory queries, and the only outputs are a scorecard and a findings report.

This is a complete, **run-it-today** package written for an administrator who has installed an MDI sensor but has never reconciled the **full** DC/AD FS/AD CS inventory against the sensor list, checked the DSA and honeytoken posture, or worked the ISPM recommendations methodically.

What you get

- A plain-language **Solution Overview** — what MDI's pieces are (sensors, the directory-service account, honeytokens, ISPM assessments, and detections), what "good" looks like, and how a read-only assessment turns that into a score.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (category-wide ones live in **Global Requirements — Assessments**).
- An **Implementation Guide** of precise, ordered steps — validate read access, reconcile sensor coverage against the AD inventory, check DSA/honeytoken/ISPM posture and detection tuning, score, generate the findings report, review, then re-run to trend.
- Five read-only automation artifacts: a **Discovery** script, an **Assess/score** script, a **per-area Coverage** check (sensor coverage the centerpiece), a **Findings Report** generator (HTML + CSV), and an access **Validate** self-check.
- **Testing** and **Troubleshooting** guides so a first run goes cleanly.

What this delivers

A defensible, point-in-time **posture score** for Defender for Identity plus a **prioritized remediation backlog**: which domain controllers / AD FS / AD CS servers have **no healthy sensor** (the critical blind spots), whether the DSA is present and hardened, whether honeytokens exist and are tagged, which ISPM

recommendations are open, and where detections are mis-tuned — each finding tagged with a severity and a fix pointer.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — this product owns the **Assess** stage. The scripts are strictly read-only; **risk-acceptance and remediation priority are the customer's security owners' decisions**, and the fixes (installing sensors, hardening the DSA, planting honeytokens) are the paired **Remediation Playbook's**, done pilot-first. Licensing and preview facts are flagged **verify** because they change.

Outcome

You stop assuming your on-prem identity is covered because MDI is "deployed." You get a scored picture and an ordered backlog — starting with the un-sensored domain controllers — the evidence to fix what matters, in the right order, and a repeatable way to re-score.

Scope boundary

This is an **assessment** — it observes, scores, and reports; it does **not** install sensors, change the DSA, or plant honeytokens. Those fixes are the paired **Remediation Playbook** (pilot-first). Unified Defender onboarding, threat hunting, and SOC operations are the existing **DEFXDR**, **THREATHUNT**, and **SOCXDR** catalog areas — this assessment cross-references them for the remediation path rather than duplicating them. **Response actions and incident determinations belong to the SOC.**

1. Solution Overview — Security Optimization Assessment: Defender for Identity

What this assessment is

This solution is a **read-only Security Optimization Assessment** for **Microsoft Defender for Identity (MDI)**. It does not install a sensor or change any configuration. It **discovers** how MDI is set up, **scores** it against Microsoft best practice, and produces a **prioritized, risk-ranked findings report**. The deliverable is *evidence and a plan*; the fixes are a separate, pilot-first exercise (the paired **Remediation Playbook**).

The problem it solves is the illusion of coverage. MDI is only watching an identity surface where a **healthy sensor** is installed — so a single un-sensored domain controller (or an AD FS / AD CS server) is a blind spot an attacker can move through undetected. The assessment reconciles what *should* be covered (the real AD inventory) against what *is* covered (the sensor list), then evaluates the supporting posture: the directory-service account, honeytokens, ISPM, and detection tuning.

How Defender for Identity fits together (what we score)

- **Sensors and coverage.** MDI sensors run on **domain controllers**, and (where present) **AD FS**, **AD CS**, and can monitor **Entra Connect** servers. Coverage is only complete when **every** such server has an **installed and healthy** sensor. *We reconcile the AD inventory against the sensor list — the single most important check.* Sensor **health** (running, up to date, not delayed) matters as much as presence.

• **Directory Service Account (DSA).** MDI uses a directory-service account to query Active Directory and enrich identity data (SAM-R, group membership, etc.). Best practice is a **group-managed service account (gMSA)** with least privilege. A missing, over-privileged, or password-based DSA is a hygiene finding. *DSA requirements evolve — verify current guidance.*

• **Honeytokens.** Decoy accounts/entities that no legitimate user should ever touch; **tagging** them in MDI means any authentication or query against them raises a high-fidelity alert. Absent or untagged honeytokens are a missed, cheap detection.

• **Identity Security Posture Management (ISPM).** MDI's posture **assessments/recommendations** (e.g. unsecure protocols, legacy configurations, risky delegations, exposed credentials) plus the identity-related **Microsoft Secure Score** recommendations quantify where the identity fabric is weak. *We read these and fold the top items into the ranked findings. Some assessments are preview — verify.*

• **Detections and tuning.** MDI raises detections across the attack kill-chain (reconnaissance, compromised credentials, lateral movement, domain dominance). Tuning matters: sensible **exclusions** (so it is not noisy) without over-excluding (so it is not blind), and correctly scoped **action accounts** for any response actions. *Response actions are write operations — out of scope here and SOC-gated.*

• **Alerts & integration signal.** Whether MDI alerts flow into Microsoft Defender XDR and are being triaged, and the volume/type of identity detections (via advanced hunting `IdentityDirectoryEvents`, `IdentityLogonEvents`, `IdentityInfo`), to rank findings by real exposure.

How the score works

Each area yields **checks** with a best-practice expectation (from the baseline named in the report — Microsoft MDI guidance + identity Secure Score, *verify current guidance*). Every check is **Pass / Partial / Fail / N-A**, and every non-pass becomes a **finding** with a **severity** (High / Medium / Low — an un-sensored DC is High), the **evidence**, and a **remediation pointer**. Areas roll up into a **posture score (%)** you can trend. The score measures; **which findings to accept, defer, or fix, and in what order, is your security owners' decision.**

What's in scope

• Read-only **discovery**: the AD inventory of domain controllers (and AD FS / AD CS / Entra Connect servers), the MDI **sensor** list and health, the **DSA** configuration signal, **honeytoken** tagging, **ISPM** recommendations, and detection/exclusion configuration where readable.

• **Sensor coverage** reconciliation — AD inventory vs. installed/healthy sensors, naming the blind DCs.

• **Scoring** against the named MDI + identity Secure Score baseline, per-check Pass/Partial/Fail.

• A **findings report** (HTML + CSV) with severities, evidence, and remediation pointers.

What's out of scope (be honest with buyers)

• **No changes.** This never installs a sensor, changes the DSA, plants a honeytoken, or tunes a detection. Those are the paired **Remediation Playbook** (pilot-first).

• **No response actions.** Disabling a user, forcing password reset, etc. are write operations and **SOC-gated** — never part of an assessment.

• **Not an MDI deployment project.** Standing up MDI and rolling out sensors is deployment work (see **DEFXDR** for unified onboarding). This assesses what exists.

• **Not threat hunting or IR.** Live hunting and incident response are **THREATHUNT** / **SOCXDR**. *IR and detection-response determinations belong to the SOC.*

• **No risk-acceptance decision.** The assessment supplies the score and evidence; accepting or prioritizing risk is the customer's.

Authentication and connections used

• **Active Directory read** (RSAT / the `ActiveDirectory` PowerShell module) as a normal **read-only** domain user — to enumerate domain controllers and locate AD FS / AD CS roles (the coverage denominator). No AD writes.

• **Microsoft Graph (read-only)** signed in as **Security Reader** / **Global Reader** — for identity **Secure Score** recommendations and **advanced hunting** (`IdentityInfo`, `IdentityDirectoryEvents`, `IdentityLogonEvents`) via `AdvancedHunting.Read.All`. *Verify current scope names.*

• **Defender portal read** for MDI sensor/health, DSA, and honeytoken state where no public read API exists — the scripts collect what is programmatically available and clearly flag portal-confirmation items as **verify**. No write scopes are ever requested.

The standard deployment model

> ***Assess the current MDI configuration and reconcile sensor coverage (discovery) → configure nothing; score and rank instead → pilot the read against one domain/site → validate read-only access and that the DC inventory reconciles with reality → expand to the whole forest, deliver the findings report, and re-run periodically to trend.*** *The repeatable pattern is* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand****; this product owns* ***Assess****, and the fixes are the Remediation Playbook's, pilot-first.*

2. Requirements — Security Optimization Assessment: Defender for Identity

The category-wide prerequisites (target tenant, least-privilege read-only access, Graph/management API access, licensing awareness, defined scope and point-in-time baseline, the best-practice reference, secure output handling, and remediation ownership) are in the separate **Global Requirements — Assessments** file — read it first and do not repeat those here. Below are only the prerequisites **specific to assessing Defender for Identity**. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. Defender for Identity is provisioned (even partially)

Description: MDI is enabled in the tenant with at least one sensor installed, so there is a configuration and sensor list to read. **Why required:** The assessment scores an existing MDI setup; a tenant with no MDI at

all is a deployment engagement, not an assessment (though the AD-inventory / coverage-gap view still has value). **How to verify:** In `security.microsoft.com` → **Settings** → **Identities**, confirm the MDI workspace and at least one sensor appear. **How to implement if missing:** If MDI is not provisioned, use the **DEFXDR** onboarding path first. Partial sensor coverage is expected — measuring the gap is the point.

R2. Read-only Active Directory access to enumerate the identity inventory

Description: The ability to read the AD forest/domains to list **all domain controllers** and locate **AD FS** and **AD CS** roles — the denominator for sensor coverage. **Why required:** Sensor coverage is only meaningful against the true count of DCs/AD FS/AD CS; MDI's own sensor list cannot reveal servers that have **no** sensor. **How to verify:** From a domain-joined host with RSAT, `Get-ADDomainController -Filter *` returns all DCs and `Get-ADForest` returns the forest. A normal domain user can read this. **How to implement if missing:** Install RSAT / the `ActiveDirectory` module; run from a domain-joined host as a read-only domain account. No elevated AD rights are needed for enumeration.

R3. Graph read access for identity Secure Score and advanced hunting

Description: Microsoft Graph read scopes for **Secure Score** (`SecurityEvents.Read.All`) and **advanced hunting** (`AdvancedHunting.Read.All`) over `IdentityInfo`, `IdentityDirectoryEvents`, `IdentityLogonEvents`. **Why required:** ISPM ranking and detection-activity signals come from these; identity Secure Score provides the posture-recommendation signal. **How to verify:** In `security.microsoft.com` → **Advanced hunting**, run `IdentityInfo | take 1`. `Connect-MgGraph` with the read scopes succeeds. **How to implement if missing:** Assign **Security Reader** / **Global Reader** and consent the read scopes. *Verify current scope names and hunting retention.*

R4. Portal read access to MDI sensor health, DSA, and honeytoken configuration

Description: Read access to **Settings** → **Identities** for sensor health, the **directory-service account** configuration, **honeytoken (tagged) entities**, and detection exclusions — some of which have no public read API today. **Why required:** Sensor health, DSA, and honeytoken posture are core checks; where no API exists, the script collects what it can and flags the rest for portal confirmation. **How to verify:** You can open **Settings** → **Identities** → **Sensors / Directory service accounts / Entity tags** in the Defender portal as the assessor. **How to implement if missing:** Assign a read-only Defender role that can view Identities settings. Record portal-confirmed values against the script's `verify` flags. *Verify current MDI settings layout and any new read APIs.*

R5. The expected server inventory is defined

Description: A documented, authoritative count/list of domain controllers (and AD FS / AD CS / Entra Connect servers) in scope. **Why required:** Coverage percentages need the real denominator; unmanaged or newly-promoted DCs are exactly the blind spots the assessment must surface. **How to verify:** You have the DC/AD FS/AD CS list from AD (R2) and can confirm it matches operational reality with the AD team. **How to implement if missing:** Reconcile the AD-derived list with the AD team before scoring so no DC is missed; `SOAMDI-Coverage.ps1` uses this as the denominator.

R6. A named best-practice baseline for scoring

Description: The reference the score is measured against — Microsoft's **Defender for Identity** deployment/best-practice guidance and the **identity Secure Score** recommendations, and/or the org standard. **Why required:** A score is only meaningful against a stated reference; the report names it so results are defensible and comparable. **How to verify:** The baseline is documented in scope and echoed in the report header. *MDI guidance evolves — verify against current Microsoft guidance at run time.* **How to implement if missing:** Adopt Microsoft's MDI guidance + identity Secure Score as the default reference; note deliberate deviations as accepted exceptions.

R7. Secure destination for the findings report

Description: A restricted location for the HTML/CSV output, which names exactly which domain controllers are un-sensored. **Why required:** A list of your identity blind spots is extremely sensitive; it must be handled on a need-to-know basis. **How to verify:** A restricted location exists and access is limited to the security owners. **How to implement if missing:** Provision a restricted location before the run; treat output as confidential. *Risk-acceptance of any finding is the customer's decision.*

Global Requirements — Assessments (category-level)

These requirements apply to every solution in the Assessments category (Security Optimization Assessments, remediation playbooks, and cross-cutting security assessments). Published once at the category level so listings stay small. Confirm these before running any assessment. Each entry: Description / Why / Verify / Implement-if-missing.

> *An assessment observes and scores; it does not change your tenant. Every assessment script here is **read-only** (the paired remediation work is separate and pilot-first). Scope, risk-acceptance, and remediation-priority decisions belong to the customer's security/compliance owners — the assessment supplies the evidence, not the authorization.*

G1. The workload/tenant being assessed

Description: An active Microsoft 365 / Entra tenant and/or Azure subscription(s) hosting the products in scope. Why: There is nothing to assess without the target environment. Verify: Sign in to the relevant portal(s) (`security.microsoft.com`, `portal.azure.com`) as an administrator. Implement: Ensure the target tenant/subscription exists and the products under assessment are deployed (even partially — assessing coverage gaps is the point).

G2. Read-only assessor access (least privilege)

Description: A read-only role sufficient to enumerate configuration — e.g. Security Reader, Global Reader, and/or Reader at the Azure scope. Why: An assessment must never require write access; read-only proves it changes nothing. Verify: Entra -> `Roles and administrators`; confirm Security Reader / Global Reader (and Azure Reader where cloud resources are assessed). Implement: Assign the read-only roles; do not grant write/admin roles for the assessment itself.

G3. Microsoft Graph / Az / management API access

Description: Microsoft Graph (Security), Azure Resource Manager, and product-specific management APIs reachable with the assessor identity. Why: The assessment scripts collect configuration and posture via these APIs. Verify: `Connect-MgGraph` (read scopes) and `Connect-AzAccount` succeed; the assessment discovery script runs. *Verify current API/permission scopes.* Implement: Consent the read-only Graph scopes; ensure network access to the Microsoft endpoints.

G4. Licensing for the products in scope

Description: The licenses/plans for the products being assessed (e.g. Defender for Endpoint/Identity/Office 365/Cloud Apps plans, Defender for Cloud plans). Why: Some posture/configuration is only present (and only assessable) when the plan is enabled; the assessment flags plan gaps too. Verify: Confirm the relevant plans in the admin/billing portal. *Verify current plan/feature mapping.* Implement: Note unlicensed products as findings (a coverage gap) rather than skipping them silently.

G5. Scope definition and a point-in-time baseline

Description: A defined assessment scope (which products, subscriptions, and users) and the understanding that results are a point-in-time snapshot. Why: Scope keeps the assessment bounded and the score comparable; posture drifts, so results are dated. Verify: The scope is documented; the assessment records the run date. Implement: Agree the scope before running; re-run periodically to trend the score.

G6. Best-practice reference baseline

Description: The best-practice / benchmark the assessment scores against (Microsoft security baselines, Secure Score, CIS/Zero-Trust as applicable). Why: A score is only meaningful against a reference; the assessment states which baseline it uses. Verify: The reference baseline is named in the findings. *Baselines evolve — verify against current Microsoft guidance.* Implement: Use the documented baseline; note where organizational policy intentionally deviates.

G7. Output handling and confidentiality

Description: A place to store the findings report and an understanding that it may contain sensitive posture detail. Why: Assessment output is a roadmap of your weaknesses; it must be handled as sensitive. Verify: A secure destination for the report exists; access is restricted. Implement: Store findings securely; share on a need-to-know basis. *Risk-acceptance of any finding is the customer's decision.*

G8. Remediation ownership (for the paired remediation)

Description: Named owners for acting on findings (the remediation is separate, pilot-first work). Why: An assessment only creates value when someone owns closing the gaps. Verify: Remediation owners are identified; the Remediation Playbook solution is available for the fixes. Implement: Assign owners; use the Remediation Playbook (pilot-first, -WhatIf-gated) to close findings and re-score.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — Security Optimization Assessment: Defender for Identity

This is the exact, ordered procedure to run the assessment end to end. It is **read-only** throughout — you install no sensor, change no DSA, and plant no honeytoken; the only artifacts you create are an assessor identity context, a scope file, and the output report. Follow the steps in order and reuse the exact object names below in later steps, testing, and troubleshooting.

Naming convention — every object you create uses `<Purpose> – <Scope> – <Stage>`:

Object	Name to use
Read-only app registration (optional, for unattended Graph reads)	`SOA-MDI-Reader – Tenant – Assess`
Scope definition file	`SOAMDI-Scope – Forest – Assess.json`
Output run folder	`SOAMDI-Run – <yyyy-mm-dd> – Assess`
Scorecard (CSV)	`SOAMDI-Scorecard – Forest – Assess.csv`
Findings report (HTML)	`SOAMDI-Findings – Forest – Assess.html`

Stage 0 — Confirm scope and baseline (Assess)

1. Agree the **in-scope forest/domains** with the owner and confirm the authoritative list of **domain controllers** and any **AD FS / AD CS / Entra Connect** servers (R5).

2. Record the **baseline** — Microsoft MDI best-practice guidance + identity Secure Score, and any org standard (R6). *Verify against current Microsoft guidance at run time.*

3. Create `SOAMDI-Scope – Forest – Assess.json` capturing: scope description, the expected DC/AD FS/AD CS counts, baseline name, and run date.

Stage 1 — Establish read-only access (Assess)

1. Confirm the assessor holds **Security Reader** / **Global Reader** in Entra, and a read-only Defender role that can view **Settings** → **Identities** (R4). Grant **no** write role. For AD, use a normal read-only domain account (R2).

2. **(Optional, for unattended Graph reads)** create the `SOA-MDI-Reader – Tenant – Assess` app registration with **read-only** Graph permissions (`SecurityEvents.Read.All`, `AdvancedHunting.Read.All`) and admin-consent. Add **no** write permissions.

3. From a domain-joined host, confirm the AD module (R2): `Get-ADDomainController -Filter * | Measure-Object`. Confirm `Connect-MgGraph` with the read scopes succeeds.

Stage 2 — Validate the read (Pilot the read)

1. Run `SOAMDI-Validate.ps1`. It confirms the identity can (a) enumerate DCs via AD read, (b) connect to Graph with read scopes, (c) run an advanced-hunting query (`IdentityInfo | take 1`) — and asserts **no write scope** is present. Fix any gap before continuing.

2. Confirm the DC count from AD reconciles with the AD team's expectation (Stage 0) — a mismatch here is a scope problem to resolve *before* measuring coverage.

Stage 3 — Discover the configuration (Assess)

1. Create the run folder `SOAMDI-Run – <yyyy-mm-dd> – Assess` (R7, restricted).

2. Run `SOAMDI-Discovery.ps1 -ScopeFile "SOAMDI-Scope – Forest – Assess.json" -OutputDir "SOAMDI-Run – <yyyy-mm-dd> – Assess"`. It collects, read-only:

• the **AD inventory** — all domain controllers, and AD FS / AD CS role holders (the coverage denominator);

• the MDI **sensor** list and health where available (via Graph/portal export; portal-confirmation items flagged **verify**);

• the **directory-service account (DSA)** signal (present? gMSA? — flagged for portal confirmation where no API);

• **honeytoken / tagged-entity** state (via advanced hunting + portal confirmation);

• **ISPM** recommendations (MDI posture assessments + identity Secure Score recommendations);

• detection **exclusions** where readable, and identity detection activity (`IdentityDirectoryEvents` volume).

All written to the run folder as JSON/CSV. Nothing is changed.

Stage 4 — Reconcile sensor coverage (Assess)

1. Run `SOAMDI-Coverage.ps1 -InputDir "SOAMDI-Run – <yyyy-mm-dd> – Assess"`. It computes the centerpiece:

• **DC sensor coverage** = DCs with an installed & healthy sensor ÷ total DCs (from AD), **naming every un-sensored or unhealthy DC** — the critical blind spots;

• **AD FS / AD CS coverage** = role servers with a sensor ÷ total;

• a **honeytoken presence** count and a **DSA present/hardened** signal.

The blind-spot server list is written explicitly for the remediation work list.

Stage 5 — Score against best practice (Assess)

1. Run `SOAMDI-Assess.ps1 -InputDir "SOAMDI-Run – <yyyy-mm-dd> – Assess" -Baseline "<MDI guidance + identity Secure Score>" -OutputCsv "SOAMDI-Scorecard – Forest – Assess.csv"`. It scores each check **Pass / Partial / Fail / N-A**, assigns a **severity** (un-sensored DC = High), rolls up a **weighted posture score (%)**, and writes the scorecard with the baseline stamped in.

Stage 6 — Generate the findings report (Validate → deliver)

1. Run `SOAMDI-Report.ps1 -InputDir "SOAMDI-Run – <yyyy-mm-dd> – Assess" -Scorecard "SOAMDI-Scorecard – Forest – Assess.csv" -OutputHtml`

`"SOAMDI-Findings – Forest – Assess.html"`. It produces the **risk-ranked HTML report** (High → Low, each with area, evidence, affected servers/count, and a remediation pointer) plus a tracking CSV. The un-sensored-DC list surfaces at the top.

2. **Validate the report reconciles** with the portal: spot-check the top findings — e.g. a DC the report flags as un-sensored should not appear (healthy) under **Settings** → **Identities** → **Sensors**; a `verify`-flagged DSA item should be confirmed against the portal.

Stage 7 — Review and hand off (Expand)

1. Walk the security owner through the report, leading with the **un-sensored domain controllers**. **Risk-acceptance and remediation priority are their decisions** — annotate accepted exceptions (e.g. a DC being decommissioned).

2. Hand the ordered backlog to the paired **Remediation Playbook** (pilot-first) for the fixes — install missing sensors, harden the DSA to a gMSA, plant/tag honeytokens, work the ISPM backlog. Make no changes from this assessment.

3. **Re-run to trend** — same scope file, new dated run folder — so coverage climbing toward 100% is provable, especially after new DCs are promoted.

5. Testing — Security Optimization Assessment: Defender for Identity

Because this is a **read-only** assessment, "testing" proves two things: the assessor has **read-only** access (and no write), and the **report is accurate** — above all that the **sensor-coverage denominator (the DC inventory) is real** and reconciles with the Defender portal. Run these gates in order; each has an explicit **pass/fail**.

Gate 1 — Read-only access proven (run `SOAMDI-Validate.ps1`)

Procedure: Run `SOAMDI-Validate.ps1` with the assessor identity from a domain-joined host. **Pass:** Success on all of — AD read enumerates domain controllers; `Connect-MgGraph` with read scopes; an advanced-hunting query (`IdentityInfo | take 1`) returns; and it confirms **no write scope** is present. **Fail:** Any read fails (fix access per R2–R4) **or** a write scope is detected (reduce to Security Reader — an assessment must not hold write access).

Gate 2 — The DC inventory is complete (the denominator)

Procedure: Compare the DC list from `Get-ADDomainController -Filter *` against the AD team's authoritative list (R5). **Pass:** The AD-derived DC list matches operational reality — every domain, every site, including recently-promoted DCs. **Fail:** DCs are missing from the enumeration (e.g. an unreachable child domain, a trust not traversed). Resolve scope/trust access first — an incomplete denominator hides the exact blind spots the assessment exists to find.

Gate 3 — Discovery completeness (run `SOAMDI-Discovery.ps1`)

Procedure: Run discovery; inspect the run folder. **Pass:** Raw outputs exist for every area — AD inventory, sensor list/health, DSA signal, honeytoken/tagged entities, ISPM recommendations, and detection activity — with `verify` flags clearly marking any item that needs portal confirmation (rather than silently omitting it). **Fail:** An area is silently empty. If sensor/DSA/honeytoken data has no API in your tenant, it must be **flagged for portal confirmation**, not dropped — confirm those values in the portal and record them.

Gate 4 — Coverage names the blind spots (run `SOAMDI-Coverage.ps1`)

Procedure: Run coverage; open the un-sensored-server list. **Pass:** DC sensor coverage is a fraction of the **AD-derived** DC count, and every un-sensored or unhealthy DC (and AD FS / AD CS server) is **named**, not just counted. Honeytoken and DSA signals are present. **Fail:** Coverage is computed against the sensor list alone (can never reveal a DC with no sensor), or blind DCs are counted but not named — re-run with the AD inventory as the denominator.

Gate 5 — Scoring is defensible (run `SOAMDI-Assess.ps1`)

Procedure: Run scoring; open the scorecard CSV. **Pass:** Every check has **Pass/Partial/Fail/N-A**, a **severity** on each non-pass (un-sensored DC = High), the **baseline name** stamped in, and the weighted posture score computes. Absent features (e.g. no AD CS in the environment) are **N-A**, not Fail. **Fail:** Checks unscored, severities missing, or an un-sensored DC scored anything other than High.

Gate 6 — Report reconciles with the portal (run `SOAMDI-Report.ps1`)

Procedure: Generate the report; spot-check the **top findings** and any `verify`-flagged item. **Pass:** Each matches the portal — a DC the report flags as un-sensored is genuinely absent (or unhealthy) under **Settings** → **Identities** → **Sensors**; the DSA/honeytoken `verify` items are confirmed. Findings sorted High → Low with evidence and remediation pointers. **Fail:** A spot-checked finding does not match — re-run discovery for a fresh snapshot, confirm the `verify` items in the portal, and never deliver an unreconciled report.

Gate 7 — Repeatability (trend)

Procedure: Re-run the full sequence against the same scope file into a new dated folder. **Pass:** Runs are comparable; sensor coverage climbing toward 100% is visible as remediation lands and new DCs are onboarded. **Fail:** Scope drifted (e.g. DCs added/removed without updating the scope file) — reconcile the inventory each run so the trend is valid.

> *These gates test the **assessment**, not the tenant's identity security. A low coverage number with named blind DCs is a *successful* assessment — it found the real gaps. The failure conditions are about the assessment's own accuracy and read-only guarantee.*

6. Troubleshooting — Security Optimization Assessment: Defender for Identity

Common issues when running the read-only assessment, each as **symptom** → **cause** → **fix**. None of these fixes change MDI configuration — they fix the *assessment run*.

`Get-ADDomainController` returns fewer DCs than expected

- **Symptom:** The AD-derived DC list is short — the coverage denominator is wrong.
- **Cause:** A child domain or trusted forest is not being traversed, DNS/RPC to a remote DC is blocked, or the run host cannot reach every domain.
- **Fix:** Run from a host with line-of-sight to all domains, specify the domains/forest explicitly (`Get-ADForest` then per-domain queries), and reconcile with the AD team (Testing Gate 2). An incomplete denominator hides exactly the blind spots you are hunting.

AD module not available

- **Symptom:** `Get-ADDomainController` is not recognized.
- **Cause:** RSAT / the `ActiveDirectory` PowerShell module is not installed on the run host.
- **Fix:** Install RSAT (or run from a domain controller/management host that has the module). A read-only domain account is sufficient (R2).

Sensor list / DSA / honeytoken data comes back empty from the script

- **Symptom:** Discovery cannot programmatically read sensor health, the DSA, or tagged entities.
- **Cause:** These MDI settings may have **no public read API** in your tenant today — they live in **Settings** → **Identities** in the portal.
- **Fix:** This is expected. The script flags these as **verify**; confirm the values in the portal and record them (Testing Gate 3). Do not treat the flag as a run failure and do not drop the item. *Verify current MDI read-API availability.*

`Connect-MgGraph` or advanced hunting fails

- **Symptom:** Graph connect errors, or `IdentityInfo | take 1` returns nothing.
- **Cause:** The Microsoft.Graph modules are missing, read scopes are not consented, or MDI is not sending identity data (no healthy sensors yet).
- **Fix:** `Install-Module Microsoft.Graph -Scope CurrentUser`; consent `SecurityEvents.Read.All` / `AdvancedHunting.Read.All` (R3). If `IdentityInfo` is genuinely empty, that itself points to a coverage problem — note it.

Coverage shows 100% but the owner suspects blind DCs

- **Symptom:** Sensor coverage looks perfect and is not believed.
- **Cause:** Coverage was computed against MDI's own sensor list (self-referential) rather than the AD-derived DC inventory — a DC with no sensor is invisible to that denominator.
- **Fix:** Re-run `SOAMDI-Coverage.ps1` using the AD inventory as the denominator (R5). The blind DCs will appear.

An un-sensored DC in the report is actually being decommissioned

- **Symptom:** A flagged DC is a known, intentional exception.
- **Cause:** The assessment cannot know operational plans.
- **Fix:** Record it as an **accepted exception** in the review (Stage 7). Risk-acceptance is the customer's decision; annotate so it is not re-flagged — but re-confirm it is actually gone on the next run.

Validate flags a write scope present

- **Symptom:** Gate 1 fails on the "no write scope" assertion.
- **Cause:** The assessor holds an admin/manage role.
- **Fix:** Reduce to **Security Reader** / **Global Reader** (and a read-only domain account) before running. An assessment must not hold write access; this is by design.

Report `verify` items not confirmed before delivery (Gate 6 risk)

- **Symptom:** The report ships with unconfirmed `verify` flags on DSA/honeytoken/sensor items.
- **Cause:** The portal-confirmation step was skipped.
- **Fix:** Before delivering, open **Settings** → **Identities** and confirm each `verify` item, updating the report. An unconfirmed `verify` is not a finding and must not be presented as one.

7. Best Practices — Security Optimization Assessment: Defender for Identity

Field-proven guidance for running this assessment cleanly, keeping it honest, and turning the output into action.

Keep it strictly read-only

- **Assess with Security Reader + a read-only domain account, never an admin role.** If the identity can install a sensor or change the DSA, the assessment is no longer provably non-disruptive. `SOAMDI-Validate.ps1` fails if a write scope is present — keep it that way.
- **Never install a sensor or change the DSA mid-assessment.** The moment you do, the point-in-time snapshot and the before/after trend are lost. Record it as a finding; fix it pilot-first in the Remediation Playbook.

The denominator is everything

- **Coverage is only as honest as the DC inventory.** MDI's own sensor list can never reveal a DC with no sensor — always reconcile against the **AD-derived** inventory, across every domain and site, including freshly-promoted DCs.
- **Name the blind DCs, don't just count them.** "92% coverage" is a headline; the value is the *named list* of the 8% an attacker could pivot through.
- **Reconcile with the AD team.** A trust you cannot traverse, or a child domain you cannot read, is itself a finding — surface it rather than silently excluding it.

Confirm what has no API — don't drop it

• **Some MDI settings (sensor health, DSA, honeytokens) may have no public read API in your tenant.** Where the script cannot read them, it flags them **verify** — confirm those in the portal and record the values. Silently omitting them produces a falsely clean report.

Interpret each area honestly

• **An un-sensored domain controller is High severity, full stop** — it is the classic identity blind spot.

• **A password-based or over-privileged DSA is a real finding** — best practice is a least-privilege **gMSA**. *Verify current DSA guidance.*

• **No honeytokens is a missed cheap win.** Tagged decoy accounts are high-fidelity tripwires; their absence is a finding, not a nicety.

• **Work the ISPM backlog, don't just admire it.** Unsecure protocols, risky delegations, and exposed credentials are the posture items attackers actually use.

• **Detection tuning cuts both ways** — over-excluding blinds MDI, under-tuning drowns the SOC. Flag both extremes.

Make the numbers defensible

• **Name the baseline and date in the report.** A percentage without its reference is not defensible.

• **Reconcile the top findings against the portal every run**, including the `verify` items — one wrong "un-sensored DC" claim undermines the whole report.

Hand off well

• **Lead the review with the un-sensored DCs** — they are the highest-impact, most-urgent fixes.

• **Let the customer own risk-acceptance.** A DC being decommissioned may be a deliberate exception; annotate it so it is not re-flagged each run.

• **Re-run on a cadence**, especially after DC promotions — show coverage climbing to 100%.

Protect the output

• The report names your identity blind spots — store it in the restricted location (R7), share on a need-to-know basis, never email it broadly.

Where to go next (adjacent catalog areas)

• **Remediation Playbook (SOAREM)** — the paired, pilot-first solution that *closes* these findings (install sensors, harden the DSA to a gMSA, plant honeytokens, work ISPM), then you re-score.

• **DEFXDR** — unified Defender onboarding if identity is part of a broader coverage gap.

• **THREATHUNT** — proactive hunting over `IdentityDirectoryEvents` / `IdentityLogonEvents` once coverage is solid.

• **SOCXDR** — SOC onboarding and operations for the identity alerts MDI generates. *Response actions and IR determinations belong to the SOC.*

• **Security Optimization Assessment — Defender for Endpoint / Office 365 / Cloud Apps / Cloud / Defender Suite** — the sibling assessments; run the **Defender Suite** assessment for a single cross-workload posture score.

8. Deliverables Summary — Security Optimization Assessment: Defender for Identity

Documentation

Document	Purpose
Executive Summary	The business case — why sensors on "most" DCs is not identity coverage, and what a read-only scored assessment delivers
1. Solution Overview	What MDI's pieces are (sensors, DSA, honeytokens, ISPM, detections), what "good" looks like, and how the score works — with sensor-coverage reconciliation the centerpiece
2. Requirements	Solution-specific prerequisites with Verify / Implement-if-missing (category-wide ones live in Global Requirements — Assessments)
Global Requirements (Assessments)	Category-wide prerequisites, published once
3. Implementation	Exact, ordered, read-only steps — validate access, reconcile sensor coverage against the AD inventory, check DSA/honeytoken/ISPM/tuning, score, generate the findings report, review, re-run to trend
5. Testing	Read-only and accuracy gates with pass/fail — proves non-disruptive and that the coverage denominator reconciles with reality and the portal
6. Troubleshooting	Common run issues → cause → fix
7. Best Practices	Keep-it-read-only, the-denominator-is-everything, confirm-what-has-no-API guidance and where to go next

Automation artifacts (all read-only)

File	What it does
`SOAMDI-Discovery.ps1`	**PowerShell (AD read + Graph), read-only.** Collects the AD inventory (all DCs, AD FS / AD CS role holders), the MDI sensor list & health where available, the DSA and honeytoken signals (portal-confirm items flagged `verify`), ISPM recommendations, and identity detection activity, into a dated run folder. Changes nothing.
`SOAMDI-Coverage.ps1`	**PowerShell, read-only.** Reconciles **sensor coverage** — DCs with a healthy sensor vs. the AD-derived total, **naming every un-sensored/unhealthy DC** — plus AD FS / AD CS coverage, honeytoken presence, and the DSA signal. The blind-spot list for remediation.
`SOAMDI-Assess.ps1`	**PowerShell, read-only.** Scores each check Pass/Partial/Fail/N-A against the MDI + identity Secure Score baseline (un-sensored DC = High), assigns severities, and rolls up a weighted posture score (%). Writes the scorecard with the baseline stamped in.
`SOAMDI-Report.ps1`	**PowerShell, read-only.** Generates the risk-ranked findings report (HTML) sorted High → Low with evidence, affected servers/counts, and remediation pointers — the un-sensored-DC list at the top — plus a tracking CSV.
`SOAMDI-Validate.ps1`	**PowerShell, read-only.** Self-check that the assessor can enumerate DCs via AD read, connect to Graph with read scopes, run an identity hunting query, **and has no write scope**. The assessment's read-only guarantee.

What the buyer receives

- This assessment guide as a **Word** document and **PDF**.
- All five read-only PowerShell scripts in a single ZIP — yours to keep and re-run to trend your coverage toward 100% over time.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — this package owns **Assess** (read-only); the fixes are the paired Remediation Playbook, done pilot-first.

Delivered as a Complete Solution Package (PDF + Word + scripts).

Outcome

A defensible, point-in-time **posture score** for Defender for Identity and a **prioritized, risk-ranked remediation backlog** — starting with the named un-sensored domain controllers — the evidence to fix what matters, in order, plus the scripts to re-score. Risk-acceptance and remediation priority remain the customer's security owners' decisions; response actions and IR determinations belong to the SOC.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SOAMDI-Assess.ps1

Purpose: Evaluates the discovery/coverage evidence as checks, each scored Pass / Partial / Fail / N-A against the named baseline (Microsoft MDI guidance + identity Secure Score, and/or the org standard). Every non-pass gets a severity (an un-sensored DC = High) and becomes a finding with evidence and a remediation pointer. Areas roll up into a weighted posture score (%). Absent roles (e.g. no AD CS in the environment) are scored N-A, not Fail. Items still needing portal confirmation carry a 'verify' note rather than a false Fail. HOW TO RUN pwsh -File .\SOAMDI-Assess.ps1 -InputDir ".\SOAMDI-Run - 2026-07-11 - Assess" -Baseline "MDI guidance + identity Secure Score (2026-07, verify)" -OutputCsv ".\SOAMDI-Scorecard - Forest - Assess.csv" Run SOAMDI-Discovery.ps1 and SOAMDI-Coverage.ps1 first. EXPECTED RESULTS A scorecard CSV + findings.json in the run folder, and a posture score to console. .NOTES White-label. Read-only. Scoring only. Risk-acceptance is the customer's decision. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, [string]$Baseline = 'Microsoft Defender for Identity guidance + identity Secure Score (verify current guidance)', [Parameter(Mandatory = $true)] [string]$OutputCsv)

How to run: pwsh -File .\SOAMDI-Assess.ps1 -InputDir ".\SOAMDI-Run - 2026-07-11 - Assess" -Baseline "MDI guidance + identity Secure Score (2026-07, verify)" -OutputCsv ".\SOAMDI-Scorecard - Forest - Assess.csv" Run SOAMDI-Discovery.ps1 and SOAMDI-Coverage.ps1 first. EXPECTED RESULTS A scorecard CSV + findings.json in the run folder, and a posture score to console. .NOTES White-label. Read-only. Scoring only. Risk-acceptance is the customer's decision. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, [string]$Baseline = 'Microsoft Defender for Identity guidance + identity Secure Score (verify current guidance)', [Parameter(Mandatory = $true)] [string]$OutputCsv)

SOAMDI-Coverage.ps1

Purpose: Reconciles what SHOULD be covered (the AD-derived inventory of domain controllers and AD FS / AD CS role holders) against what IS covered (the MDI sensor list), and NAMES every un-sensored or unhealthy server — the critical blind spots. Also reports honeytoken presence and the DSA signal. Coverage is computed against the AD inventory as the denominator (never the sensor list alone, which can't reveal a server with no sensor). HOW TO RUN pwsh -File .\SOAMDI-Coverage.ps1 -InputDir ".\SOAMDI-Run - 2026-07-11 - Assess" Run SOAMDI-Discovery.ps1 first. Where sensor data was 'verify'-flagged (no API), confirm the sensor list in the portal and, if needed, supply it via -SensorHostsCsv for exact reconciliation. EXPECTED RESULTS A coverage summary + coverage-blindspots.csv in the run folder. Read-only. .NOTES White-label. Read-only. Consumes discovery output only; makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, Optional: a CSV/txt of sensor host names confirmed from the portal, one per line, when no API is available. [string]$SensorHostsCsv = '')

How to run: pwsh -File .\SOAMDI-Coverage.ps1 -InputDir ".\SOAMDI-Run - 2026-07-11 - Assess" Run SOAMDI-Discovery.ps1 first. Where sensor data was 'verify'-flagged (no API), confirm the sensor list in the portal and, if needed, supply it via -SensorHostsCsv for exact reconciliation. EXPECTED RESULTS A coverage summary + coverage-blindspots.csv in the run folder. Read-only. .NOTES White-label. Read-only. Consumes discovery output only; makes no changes. > [CmdletBinding()] param(

[Parameter(Mandatory = $true)] [string]$InputDir, Optional: a CSV/txt of sensor host names confirmed from the portal, one per line, when no API is available. [string]$SensorHostsCsv = '')

SOAMDI-Discovery.ps1

Purpose: Builds the raw evidence base the coverage and scoring passes consume. Collects, read-only: the AD inventory (all domain controllers, and AD FS / AD CS role holders) as the coverage denominator; the MDI sensor list & health where programmatically available; the directory-service account (DSA) and honeytoken/tagged-entity signals (flagged 'verify' for portal confirmation where no public read API exists); ISPM recommendations (MDI posture assessments + identity Secure Score); and identity detection activity via advanced hunting. Nothing is changed. HOW TO RUN pwsh -File .\SOAMDI-Discovery.ps1 -ScopeFile ".\SOAMDI-Scope - Forest - Assess.json" -OutputDir ".\SOAMDI-Run - 2026-07-11 - Assess" Run SOAMDI-Validate.ps1 first, from a domain-joined host with the ActiveDirectory module. Verify current MDI read-API availability and Graph scope names.* EXPECTED RESULTS A populated run folder with one file per area plus discovery-manifest.json; items with no API are flagged 'verify' for portal confirmation. Read-only — no changes. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$ScopeFile, [Parameter(Mandatory = $true)] [string]$OutputDir)

How to run: pwsh -File .\SOAMDI-Discovery.ps1 -ScopeFile ".\SOAMDI-Scope - Forest - Assess.json" -OutputDir ".\SOAMDI-Run - 2026-07-11 - Assess" Run SOAMDI-Validate.ps1 first, from a domain-joined host with the ActiveDirectory module. Verify current MDI read-API availability and Graph scope names.* EXPECTED RESULTS A populated run folder with one file per area plus discovery-manifest.json; items with no API are flagged 'verify' for portal confirmation. Read-only — no changes. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$ScopeFile, [Parameter(Mandatory = $true)] [string]$OutputDir)

SOAMDI-Report.ps1

Purpose: Turns the scorecard, coverage, and blind-spot list into a deliverable: a risk-ranked HTML report (findings sorted High -> Low with area, description, evidence, and remediation pointer), the headline posture score, the coverage summary, and — surfaced prominently at the top — the list of un-sensored / unhealthy domain controllers and role servers. Plus a companion tracking CSV. This is the artifact the security owner reviews and hands to the Remediation Playbook. HOW TO RUN pwsh -File .\SOAMDI-Report.ps1 -InputDir ".\SOAMDI-Run - 2026-07-11 - Assess" -Scorecard ".\SOAMDI-Scorecard - Forest - Assess.csv" -OutputHtml ".\SOAMDI-Findings - Forest - Assess.html" Run SOAMDI-Assess.ps1 first. Confirm any 'verify' items in the portal before delivery. EXPECTED RESULTS An HTML findings report and findings-tracking.csv. Read-only — reads run output only. .NOTES White-label. Read-only. Store output securely — it names your identity blind spots. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, [Parameter(Mandatory = $true)] [string]$Scorecard, [Parameter(Mandatory = $true)] [string]$OutputHtml, [string]$Title = 'Security Optimization Assessment — Defender for Identity')

How to run: pwsh -File .\SOAMDI-Report.ps1 -InputDir ".\SOAMDI-Run - 2026-07-11 - Assess" -Scorecard ".\SOAMDI-Scorecard - Forest - Assess.csv" -OutputHtml ".\SOAMDI-Findings - Forest - Assess.html" Run SOAMDI-Assess.ps1 first. Confirm any 'verify' items in the portal before delivery. EXPECTED RESULTS An HTML findings report and findings-tracking.csv. Read-only — reads run output only. .NOTES White-label. Read-only. Store output securely — it names your identity blind spots. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$InputDir, [Parameter(Mandatory =

$true)] [string]$Scorecard, [Parameter(Mandatory = $true)] [string]$OutputHtml, [string]$Title = 'Security Optimization Assessment — Defender for Identity')

SOAMDI-Validate.ps1

Purpose: The read-only guarantee of the assessment. Confirms the current identity can (a) enumerate domain controllers via read-only Active Directory queries (the coverage denominator), (b) connect to Microsoft Graph with read scopes, and (c) run an identity advanced-hunting query (IdentityInfo | take 1) — and asserts that NO write scope is present on the token. Run from a domain-joined host with the ActiveDirectory module and a read-only domain account. HOW TO RUN pwsh -File .\SOAMDI-Validate.ps1 Requires RSAT / ActiveDirectory module and Microsoft.Graph modules. Prompts for interactive read-only consent if needed. *Verify current Graph scope names and MDI read-API availability.* EXPECTED RESULTS A PASS/FAIL line per check and an overall verdict. Read-only — makes no changes. .NOTES White-label. Read-only. Makes no changes. Assessment access self-check. > [CmdletBinding()] param([string[]]$ReadScopes = @('SecurityEvents.Read.All'))

How to run: pwsh -File .\SOAMDI-Validate.ps1 Requires RSAT / ActiveDirectory module and Microsoft.Graph modules. Prompts for interactive read-only consent if needed. *Verify current Graph scope names and MDI read-API availability.* EXPECTED RESULTS A PASS/FAIL line per check and an overall verdict. Read-only — makes no changes. .NOTES White-label. Read-only. Makes no changes. Assessment access self-check. > [CmdletBinding()] param([string[]]$ReadScopes = @('SecurityEvents.Read.All'))